



Gymnázium J. K. Tyla

Úvod do agentického AI a harness pro automatizovaný softwarový vývoj

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Tato odborná práce se zabývá principy agentního inženýrství (*agentic engineering*) a architekturou řídicích harnessů pro automatizovaný vývoj softwaru. Praktickým přínosem práce je návrh a implementace systému DarkFactory — agentního harnessu instalovatelného jako aplikace pro platformu GitHub (GitHub App). Systém usiluje o maximální možnou míru automatizace vývojového cyklu od sémantické analýzy požadavků v GitHub Issues, přes technické plánování, až po generování kódu a vystavení pull requestu. Práce reflektuje, že současné agentní systémy nelze vnímat jako plně autonomní: jazykové modely vyžadují deterministické mantinely proti uvíznutí v nekonečných cyklech, správu kontextu bez sémantického posunu a především kontinuální zapojení člověka formou schvalovacích bran (*Human-in-the-loop*).

Klíčová slova

agentní inženýrství, řídicí harness, GitHub App, DarkFactory, automatizace vývoje, orchestrace agentů, jazykové modely

Annotation

This thesis explores the foundational principles of agentic engineering and harness architecture for automated software development. The practical contribution of the work is the design and implementation of DarkFactory — an agentic harness installable as a GitHub App. The system aims to maximize the automation of the software engineering lifecycle, ranging from issue interpretation in GitHub Issues and technical planning to code generation and pull request delivery. The thesis emphasizes that modern agentic systems cannot be considered fully autonomous: large language models require deterministic guardrails against infinite loops, rigorous context management without semantic drift, and indispensable human-in-the-loop governance via approval gates.

Keywords

agentic engineering, harness architecture, GitHub App, DarkFactory, software development automation, agent orchestration, language models

Obsah

Prohlášení	2
Anotace	3
Klíčová slova	3
Annotation	3
Keywords	3
1 Úvod	6
1.1 Motivace a vymezení problému	6
1.2 Cíl práce a výzkumné otázky	6
1.3 Metodika práce	7
2 Teoretická část	8
2.1 Řízení verzí a kontinuální integrace	8
2.2 Jazykové modely a dynamika kontextového okna	9
2.3 Architektura řídicího harnessu a orchestrace	11
2.4 Zapojení člověka do smyčky (Human-in-the-loop)	14
3 Praktická část	16
3.1 Cíl a rozsah systému DarkFactory	16
3.2 Architektura a životní cyklus požadavku	16
3.3 Orchestrační jádro a validační infrastruktura	17
3.4 Systém revizních značek pro lidský dohled nad akademickým textem	18
4 Výsledky a diskuse	19
4.1 Sjednocení pracovních postupů a přenositelnost napříč doménami	19
4.2 Provozní metriky a spolehlivost agentních běhů	20
4.3 Poznatky a systémová úskalí z provozu	20
4.4 Diskuse: Porovnání architektur	20
4.5 Systémová a metodická omezení	21
5 Závěr	22
Seznam příloh	23
A Obsah přiloženého média	24
B Schéma konfiguračního manifestu darkfactory.json	25
C Sdílené workflow pro GitHub Actions	26
D Systémové prompty plánovacího a kódovacího agenta	27
E Protokol revizních značek v sazebním systému Typst	28
E.1 Vizualní reprezentace jednotlivých prvků v sazbě	28
Seznam zdrojů	30

Seznam obrázků a tabulek

Obrázek 1	Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.	12
Tabulka 1	Rozpoznávání prostředí podle souboru popisujícího balíček.	17

1 Úvod

1.1 Motivace a vymezení problému

Vývoj softwaru se za posledních dvacet let z velké části zautomatizoval. Sestavení programu, spuštění testů, kontrola stylu i nasazení do provozu dnes obstarávají stroje a nikdo je nepovažuje za práci hodnou lidského času. Jeden krok však dlouho zůstával výhradně ruční: samotná změna zdrojového kódu. Právě tam vzniká většina časových prodlev — v intervalu mezi okamžikem, kdy vývojář či zákazník formuluje požadavek, a okamžikem, kdy se otestovaná změna dostane k uživatelům.

Výrobní průmysl pro vysoce automatizované provozy zavedl pojem *dark factory* („temná továrna“) — výrobní halu pracující bez lidské obsluhy, v níž není nutné svítit. Smyslem tohoto konceptu není naprosté vytlačení člověka (i temná továrna vyžaduje inženýry a konstruktéry, kteří určují, co a proč se má vyrábět), nýbrž eliminace člověka z mechanicky se opakujících úkonů.

S příchodem velkých jazykových modelů se otevřela cesta k uplatnění téhož principu v softwarovém inženýrství. Většina současných nástrojů však řeší pouze izolovaný krok: vygenerují návrh změny kódu, který musí člověk manuálně zasadit do kontextu, spustit v lokálním prostředí a vyřešit případné syntaktické regrese. V teorii i praxi dosud chybí ucelený popis toho, jak má autonomní agent zapadnout do celého vývojového cyklu — kdo a na základě jakých podkladů schvaluje jeho záměr, jaké deterministické záruky zabrání rozpadu procesu při selhání modelu a jakým způsobem se objektivně ověřuje správnost výsledku. Předmětem této práce proto není triviální otázka, zda jazykový model dokáže napsat fragment kódu, nýbrž otázka, jaká kontrolní architektura (harness) musí model obklopot, aby jeho výstupům bylo možné v produkčním repozitáři důvěřovat.

1.2 Cíl práce a výzkumné otázky

Cílem práce je představit teoretické principy agentního inženýrství (*agentic engineering*) a navrhnout architekturu řídicího harnessu pro automatizovaný softwarový vývoj, který provede vývojový požadavek celým životním cyklem od zadání po ověřenou změnu, aniž by se vzdal lidského dohledu v klíčových rozhodovacích bodech.

Dílčí cíle:

1. Popsat současný stav automatizace softwarového vývoje a vymežit deterministické základy (správu verzí a kontinuální integraci).
2. Analyzovat principy agentního inženýrství a limity velkých jazykových modelů (dynamika kontextového okna, jev Context Rot, ztrátová komprese a sémantický posun).

3. Navrhnout modulární architekturu řídicího harnessu zahrnující nástrojové smyčky (ReAct) s deterministickou detekcí uvíznutí, rozpočtem tahů a bezpečnostními pískovišti.
4. Vymezit mechanismy zapojení člověka do smyčky (*Human-in-the-loop*), formalizovat schvalovací brány a navrhnout protokol revizních značek pro dohled nad textovými výstupy.

V návaznosti na stanovené cíle práce zkoumá tři klíčové inženýrské výzkumné otázky:

- **VO1 (Míra automatizace a role člověka):** Lze vývojový proces od zadání požadavku (GitHub Issue) po vytvoření funkčního pull requestu strukturovat tak, aby role vývojáře spočívala v architektonickém dozoru a schvalování záměru (Human Gate), aniž by musel sám psát kód nebo řešit syntaktické regrese?
- **VO2 (Řízení divergence a spolehlivost smyčky):** Jakými deterministickými mechanismy (stuck detection, rozpočet tahů, circuit breaker) lze v řídicím harnessu zabránit patologiím jazykových modelů, jako je perseverace, oscilace či nekonečné zacyklení v ReAct smyčce?
- **VO3 (Integrita paměti a eliminace sémantického posunu):** Jakými postupy lze efektivně spravovat kontextové okno agenta při komplexních úlohách, aby nedocházelo k degradaci pozornosti (Context Rot) a destruktivní ztrátě architektonických invariantů při rekurzivní kompresi?

1.3 Metodika práce

Práce je z povahy tématu teoreticko-architektonická a inženýrská: primárním výstupem je formulace principů agentního inženýrství a návrh robustní architektury řídicího harnessu pro automatizovaný softwarový vývoj. Postup odpovídá inženýrskému cyklu:

1. **Koncepční analýza:** Systematické zmapování limitů autoregresivních modelů, dynamiky kontextového okna, jevu Context Rot a rozhraní nástrojů.
2. **Architektonický návrh:** Formulace modulárního modelu řídicího harnessu zahrnujícího správu stavu, exekuční pískoviště (sandbox), bezpečnostní pojistky a orchestraci subagentů.
3. **Kritické zhodnocení:** Analýza navržených principů ve srovnání se současnými monolitickými agentními smyčkami a formulace provozních limitů autonomního inženýrství.

2 Teoretická část

2.1 Řízení verzí a kontinuální integrace

Základním stavebním kamenem deterministické infrastruktury pro autonomní vývoj softwaru je systém správy verzí (VCS) úzce provázaný s mechanismy kontinuální integrace. Distribuovaný model správy verzí, jehož nejrozšířenějším zástupcem je Git (1), formálně reprezentuje repozitář jako orientovaný acyklický graf (*Directed Acyclic Graph*, DAG). V tomto grafu tvoří uzly neměnné objekty revizí (*commits*) provázané kryptografickými hashy (SHA-1 či SHA-256) na své předchůdce, zatímco větve představují odlehčené, pohyblivé ukazatele (*refs*) na konkrétní uzly grafu. Každý přispěvatel má k dispozici úplnou lokální kopii historie, což umožňuje provádět i zkoumat změny nezávisle a slučovat je až ve chvíli jejich ověření.

Tato architektura poskytuje klíčové operační mechanismy pro zapojení autonomních agentů:

- **Izolace stavu ve větvích:** Agent operuje ve vyhrazené pracovní větvi (např. `feature/...` nebo `agent/...`) odbočující z hlavní linie (`main`). Veškeré pokusné mutace souborového systému, mezistavy a ladicí iterace zůstávají striktně izolované, aniž by ohrožily stabilitu produkčního kódu nebo práci lidských spolupracovníků.
- **Deterministický audit:** Každý krok agenta lze zachytit jako atomický commit s přesným časovým otiskem, autorskými metadaty a odkazem na kontextové zadání, což vytváří neměnnou a zpětně reprodukovatelnou historii změn.
- **Strategie slučování:** Při integraci pracovní větve se uplatňují různé topologické strategie (přímý posun ukazatele či explicitní slučovací uzel). V agentických pipeline je preferováno sloučení celé sekvence dílčích mezikroků do jediného čistého uzlu (*squash and merge*), které eliminuje šum v podobě neúspěšných pokusů modelu a v hlavní větvi zanechává pouze finální, ověřený přírůstek.
- **Model pull requestu:** Sloučení větve do chráněné hlavní linie se v moderním inženýrství odehrává prostřednictvím modelu **pull requestu** (PR, na platformě GitLab též *Merge Request*). Jde o formalizovaný procesní uzel předkládající navržený diff kódu k revizi. V agentickém vývoji plní PR nezastupitelnou roli lidské schvalovací brány (*Human Gate*): poskytuje přehledné rozhraní zobrazující řádkový diff, výsledky automatických testů a strukturovaný popis úprav, v němž člověk provádí finální sémantickou kontrolu dle principu *Human-in-the-loop*.
- **Kontinuální integrace:** Na otevření či aktualizaci pull requestu bezprostředně reaguje integrační server. Kontinuální integrace (angl. *continuous integration*) představuje praxi, při níž se každá změna v malých dávkách automaticky sestaví a otestuje v izolovaném

prostředí (2), takže odchylky jsou zachyceny bezprostředně u svého vzniku. Vzhledem ke stochastické povaze jazykových modelů plní CI roli deterministického arbitra správnosti — stochastický generátor nekompromisně podrobuje exekutivní verifikaci.

- **Požadované kontroly:** Ke kontinuální integraci patří pojem **požadovaných kontrol** (*required checks*) — množina automatických úloh (linting, statická typová analýza, jednotkové a integrační testy), které musí skončit úspěchem, jinak řídicí systém sloučení změny zablokuje. Z kvality se tak stává strojově vynucovaná vlastnost. Zásadním úskalím návrhu těchto kontrol je ošetření podmíněčných běhů: úloha, která se za určitých okolností přeskočí a neohlásí žádný výsledek, může chráněnou větev trvale zablokovat. Každá kontrola proto musí deterministicky skončit explicitním výstupem (např. konstatováním, že pro daný typ souborů není definována žádná akce).
- **Sestavení a artefakty:** Výsledkem úspěšného průchodu integrační pipeline bývá reprodukovatelný **artefakt** (spustitelný binární balíček, knihovna nebo vysázený dokument). Automatizace vydávání verzí propojuje vygenerovaný artefakt s konkrétní značkou (*tagem*) v historii gitu, čímž ke každé publikované verzi garantuje doložitelný a dohledatelný výstup.

2.2 Jazykové modely a dynamika kontextového okna

Základním stavebním kamenem moderních kódovacích systémů jsou velké jazykové modely (*Large Language Models*, LLM) vystavěné na architektuře dekodérového transformeru (*Decoder-only*, např. řady Claude, GPT, LLaMA či DeepSeek) (3). V agentním inženýrství model nevystupuje jako vševědoucí orákulum, nýbrž jako **stochastický autoregresivní generátor**: na základě zadané textové historie (kontextu) opakovaně predikuje nejpravděpodobnější následující symboly. Tyto symboly reprezentují přirozený jazyk, programovací kód nebo strukturované instrukce pro vyvolání nástrojů.

Jazykový model neoperuje přímo s textovými znaky, nýbrž s diskrétními jednotkami zvanými **tokeny**. Převod mezi surovým textem a identifikátory tokenů provádí deterministický **tokenizér** (pracující např. s algoritmem Byte-Pair Encoding). Před samotným výpočtem neuronové sítě je každý token převeden do vícerozměrného vektorového prostoru — tzv. **embeddingu**, který přiřazuje slovům geometrické souřadnice podle jejich sémantické příbuznosti. Ačkoliv moderní multimodální modely (*Vision-Language Models*, VLM) dokážou transformovat i obrazové vstupy na vizuální tokeny (např. pro interpretaci wireframů), v agentním inženýrství zůstávají autonomní kódovací systémy primárně textové — staví na práci se zdrojovým kódem, abstraktními syntaktickými stromy (AST), diffy v Gitu a textovými výpisy nástrojů.

Z vlastností tokenizace a architektury transformeru plynou tři zásadní inženýrské aspekty:

- **Kapacitní strop a dynamika tahů (*Turns*):** Každému diskrétnímu kroku ve výměně informací mezi okolním prostředím a modelem se říká tah (*turn*). Agentní smyčka rozlišuje tah uživatele či prostředí (*User/Environment Turn*), tah modelu (*Model Turn*) emitující odpověď či volání nástroje, a syntetický tah vykonání nástroje (*Tool Execution Turn*) vracející výsledek z harnessu. Striktní serializace těchto tahů do historie umožňuje inferenčnímu enginu cachovat mezivýpočty pozornosti klíčů a hodnot (*KV cache*) a nepočítat historii od začátku.
- **Kvadratická složitost kontextového okna:** Model je schopen pojmout pouze omezený objem vstupu — tzv. kontextové okno (*Context Window*). Velikost okna je limitována hardwarovou náročností mechanismu pozornosti ($O(N^2)$ vzhledem k délce sekvence N) a velikostí alokované KV cache v operační či grafické paměti.
- **Jazyková asymetrie a náklady:** Výpočetní náročnost i cena komerčních API se účtují za počet zpracovaných tokenů. Texty v češtině spotřebovávají kvůli bohaté flexi a diakritice 2× až 3× více tokenů než ekvivalentní text v angličtině. V autonomních pipeline je proto vysoce optimální udržovat vnitřní systémové prompty, technické plány a strukturované logy harnessu v angličtině.

Při dlouhotrvajících úlohách naráží agent na vyčerpání kapacity kontextu. K řešení tohoto problému slouží komprese historie (**Compaction**), kdy řídicí harness vyzve model k vytvoření syntetického souhrnu dosavadního průběhu sezení, který v okně nahradí původní detailní historii kroků.

Z hlediska systémového inženýrství však textová rekurzivní komprese představuje **destruktivní ztrátovou kompresi**. Autoregresivní model při rekurzivním zkracování podléhá konfirmačnímu zkreslení (*confirmation bias*) a preferuje fakta odpovídající jeho vnitřním statistickým asociacím. Tato ztrátovost vyvolává tři kritické patologie:

1. **Vymazání deterministických detailů:** Nevratně zanikají přesná čísla řádků, signatury privátních funkcí, přesné cesty v souborovém systému a doslovná znění chybových hlášení kompilátoru či testů.
2. **Ztráta negativních omezujících podmínek:** Zvláště náchylné k vymizení jsou explicitní zákazy stanovené uživatelem v úvodu úlohy (např. zákaz modifikace veřejného API nebo zákaz přidávání externích knihoven). V syntetickém souhrnu bývají tyto podmínky zobecněny či zcela opomenuty, což vede k jejich následnému porušení.
3. **Sémantický posun (*Semantic Drift*):** Při vícenásobné rekurzivní kompresi dochází ke kaskádovému kumulování chyb (analogii tiché pošty): pokud je stav S_{k+1} generován ze souhrnu S_k a nového přírůstku Δ_k , drobné halucinace z kola k jsou v kole $k + 1$ přijaty jako nezvratná historická fakta. Po několika cyklech se model reality agenta zcela rozejde se skutečným stavem repozitáře.

K překonání těchto limitů moderní architektury uplatňují tři rigorózní alternativy:

- **Hierarchická epizodická paměť (RAG):** Kompletní doslovná historie tahů a výpisů nástrojů se ukládá do externí databáze a do aktivního okna se selektivně injektují pouze fragmenty bezprostředně relevantní pro aktuální podúkol.
- **Persistentní graf stavu projektu (Project State Graph):** Harness udržuje explicitní strukturovaný stav projektu (modifikované soubory, otevřené otázky, výsledky testů, architektonické invarianty), jehož aktualizace se řídí přísným schématem a nepodléhá narativní degradaci.
- **Selektivní prořezávání KV cache (Selective KV Cache Eviction):** Algoritmy jako StreamingLLM (4) či H_2O (5) uvolňují paměť přímo na úrovni tenzorů v inferenčním enginu se zachováním klíčových kotev pozornosti (*attention sinks*) bez nutnosti přepisovat text promptu.

S délkou kontextu souvisí též jev **Context Rot** — degradace schopnosti modelu věnovat rovnoměrnou pozornost celé historii sezení (jev *Lost in the Middle* (6)). Ačkoliv na syntetickém testu *Needle In A Haystack* (NIAH — nalezení izolovaného faktu v dlouhém textu) dosahují moderní modely vysoké úspěšnosti, reálný softwarový vývoj vyžaduje tzv. **Multi-Needle Reasoning** — schopnost současně nalézt, propojit a logicky syntetizovat několik vzájemně závislých faktů napříč soubory a testy. V tomto scénáři spolehlivost křížového uvažování s rostoucím zaplněním okna prudce klesá.

Závěrečným pilířem práce s modelem je **inženýrství promptů a kontextu** (7):

- **Systémový prompt:** Základní direktiva vymezující identitu agenta, dostupné nástroje a mantinely (formát commitů, zákaz destruktivních operací).
- **Few-shot prompting a Chain-of-Thought:** Ukázky vzorových řešení v kontextu a vedení modelu k explicitní formulaci mezikroků rozvahy před generováním kódu.
- **Dynamická injekce kontextu:** Průběžné doplňování reálného stavu repozitáře, stromu souborů a chybových výpisů kompilátoru.
- **Úskalí negativních instrukcí:** Jazykové modely často ignorují zákazy formulované negací (např. „nemazat existující testy“). Matematická podstata pozornosti (QK^T) totiž aktivuje vektorové asociace ke slovům zmíněným v záporu dříve, než autoregresivní proces uplatní logický operátor negace. V agentním inženýrství se proto uplatňuje kombinace **afirmativní formulace pravidel** (pozitivní vymezení povolených mantinelů) a **deterministické exekuční ochrany v harnessu** (např. připojení chráněných souborů pouze pro čtení a blokáce v CI).

2.3 Architektura řídicího harnessu a orchestrace

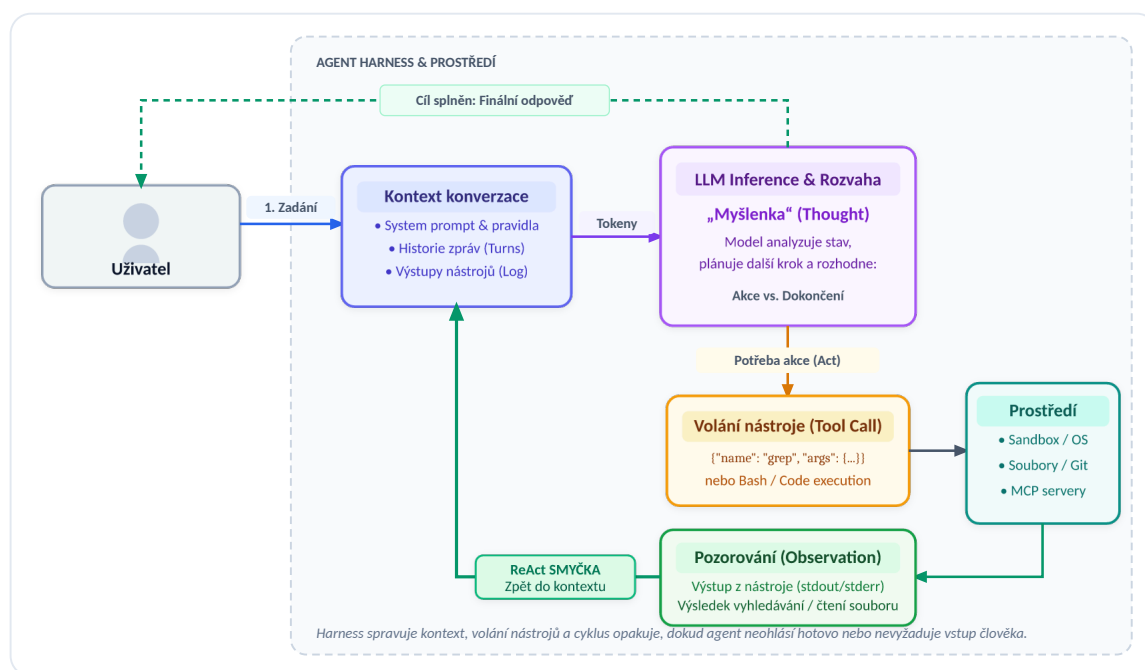
Fundamentální rozdíl mezi konverzačním chatbotem a autonomním agentem spočívá v rozsahu interakce s prostředím: zatímco chatbot generuje pouze textové odpovědi v uzavře-

ném rozhraní, agent je vybaven sadou výkonných nástrojů (*tools*), jimiž dokáže aktivně zkoumat repozitář, modifikovat soubory a spouštět příkazy.

Řídící program obklopující model se nazývá **harness**. Odlišuje se od samotného inferenčního jádra (*inference engine*), jež zajišťuje maticové výpočty sítě: harness spravuje sezení, sestavuje systémový prompt, zajišťuje bezpečné spouštění nástrojů a řídí iterativní smyčku uvažování.

Základním prováděcím cyklem agenta je smyčka **ReAct** (*Reasoning and Acting*) (8). Jak znázorňuje diagram na Obrázek 1, po přijetí zadání probíhá iterativní cyklus čtyř fází:

1. **Fáze rozvahy (Thought)**: Model v rámci inference analyzuje stav kontextu a formuluje myšlenkový postup pro nejbližší krok.
2. **Vyvolání akce (Tool Call)**: Model emituje strukturovaný požadavek na volání nástroje.
3. **Vykonání v harnessu a pozorování (Observation)**: Harness požadavek bezpečně provede v cílovém prostředí a výstup vloží do kontextu jako nové pozorování.
4. **Navazující iterace**: Aktualizovaný kontext je předložen modelu v dalším tahu, čímž agent adaptivně reaguje na reálnou odezvu prostředí.



Obrázek 1: Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.

Předpoklad, že smyčka ReAct přirozeně konverguje k vyřešení problému, v reálném softwarovém inženýrství často selhává. Jazykový model podléhá patologickým stavům divergence:

- **Perseverance a zacyklení**: Model opakovaně emituje identické volání nástroje se shodnými argumenty (např. čte neexistující soubor či spouští tentýž selhávající test), a to i po obdržení chybové odezvy. V kontextu vzniká pravděpodobnostní atraktor, který nutí model k opakování chybného tokenového sledu.

- **Oscilace a těkání (*Thrashing*):** Model střídavě přepíná mezi dvěma protichůdnými zásahy (úprava modulu A rozbije modul B a následná oprava B rozbije A).
- **Nekontrolovaná spotřeba zdrojů (*Context Runaway*):** Rychlé vyčerpání kontextového okna i rozpočtu na API volání.

Řídící harness proto musí působit jako deterministický dozorčí mechanismus (*watchdog*) a vynucovat následující ochrany:

- **Rozpočet tahů a nákladů (*Step & Cost Budget*):** Stanovení pevného limitu tahů $T_{\max}$ (typicky 25–50 kroků) a finančního stropu tokenů, jejichž překročení vede k bezpodmínečnému zastavení inference.
- **Algoritmická detekce uvíznutí (*Stuck Detection*):** Výpočet kanonického hashe volání nástroje:

$$h_t = \text{hash}(\text{nástroj}, \text{canonicalize}(\text{argumenty}))$$

Harness v klouzavém okně posledních k tahů detekuje identické volání $h_t = h_{t-1} = \dots = h_{t-k+1}$ nebo cyklické periody.

- **Dvoustupňová intervence:** Při první detekci repetice harness injektuje prioritní syntetické varování rozbíjející pravděpodobnostní atraktor v matici pozornosti. Pokud perseverace pokračuje, zasáhne tvrdý **circuit breaker** — okamžité ukončení cyklu, deterministický návrat (*git rollback*) neuložených změn na poslední stabilní commit a eskalace člověku.

K interakci s prostředím slouží dva přístupy: strukturované volání nástrojů (Function / Tool Calling v JSON) a přímé spouštění kódu (Code Execution) v sandboxu. Zatímco JSON volání je spolehlivé pro atomické operace, pro komplexní vývoj přináší formátovací režii tokenů. Přímé spouštění kódu v kontejneru nabízí maximální flexibilitu, avšak přináší závažná bezpečnostní rizika.

Modularitu a rozšiřitelnost harnessu zajišťují tři zavedené koncepty:

- **Dovednosti (*Skills*):** Adresáře instrukcí a referencí se souborem `SKILL.md` využívajícím YAML frontmatter. Do základního systémového promptu se načítají pouze metadata; detailní návod se načte až při explicitním vyvolání, což šetří kontextové okno.
- **Skripty a záchytné body (*Scripts & Hooks*):** Deterministické spustitelné programy pro opakující se transformace kódu a událostní háčky vyvolávané při stavových přechodech harnessu.
- **Model Context Protocol (MCP (9)):** Otevřený standard sjednocující integraci externích nástrojů a datových zdrojů přes protokol JSON-RPC (prostřednictvím `stdio` nebo `SSE`), který striktně odděluje běhové prostředí agenta od implementace nástrojů.
- **Meta Harness:** Koncept samořízené evoluce harnessu samotným agentem (10). Ačkoliv je vysoce efektivní pro osobní agenty maximalizující vlastní autonomii, pro komerční

a podnikové nasazení je vyžadováno deterministické ohraničení vylučující svévolnou modifikaci systémových pravidel.

Při řešení rozsáhlých softwarových problémů lineární smyčka jediného agenta naráží na kapacitní stropy. Systém proto využívá dvě pokročilé orchestrační techniky:

- **Subagenti (*Subagents*):** Hlavní orchestrátor hierarchicky dekomponuje problém a pověřuje specializované subagenty izolovanými úkoly (např. průzkum repozitáře, řešerše dokumentace). Po dokončení úkolu je kontext subagenta zahozen a orchestrátoru je předán pouze čistý výsledek, což chrání primární kontextové okno před zahlcením (*context pollution*).
- **Pracovní postupy jako grafy (Graph Engineering / DAG):** Životní cyklus požadavku je formalizován jako orientovaný acyklický graf (DAG), kde uzly tvoří specializované fáze (detekce, plánování, implementace, verifikace, schválení) a hrany definují tok dat a závislosti (*needs*). Selhání v kterémkoli uzlu okamžitě zastaví navazující kroky, což garantuje determinismus a stabilitu repozitáře.

2.4 Zapojení člověka do smyčky (Human-in-the-loop)

Čím vyšší míru samostatnosti systém vykazuje, tím kritičtější je přesné vymezení bodů, v nichž do procesu vstupuje člověk. Cílem moderního inženýrství není nekritická plná autonomie, nýbrž automatizace rutinních a mechanických kroků v kombinaci s lidským rozhodnutím tam, kde je změna nevratná nebo kde chybí jednoznačné algoritmické měřítko správnosti.

Architektura zapojení člověka do smyčky (*Human-in-the-loop*) stojí na třech principech:

- **Schvalovací body a lidské brány (Human Gates):** Schvalovací bod představuje formální procesní uzel, v němž se exekuce zastaví a vyčká na autorizaci operátora. Klíčovým inženýrským rozhodnutím je správná granularita: příliš časté dotazování vede k únavě z revizí (*review fatigue*) a ztrátě efektivity, zatímco absence kontrol znamená ztrátu kontroly. Optimální architekturou je dvoufázové schvalování: člověk autorizuje **záměr** (analýzu zadání) a **technický plán** dříve, než agent začne generovat kód v repozitáři, a následně provede finální sémantickou kontrolu diffu v pull requestu.
- **Dohledatelnost původního zadání:** Aby byla změna auditovatelná, musí systém trvale uchovávat doslovné znění původního požadavku (např. znění GitHub Issue). Jakákoli předčasná parafráze modelem vede ke ztrátě sémantických nuancí a omezujících podmínek vložených autorem zadání.
- **Transparence selhání a eskalace:** Systém, který chyby tlumí či zamlžuje halucinovanými omluvami, postupně eroduje důvěru uživatele. Pokud agent vyčerpá rozpočet tahů, naráží na detekci uvíznutí nebo opakovaně selže při opravě testů, harness nesmí tíše

skončit. Místo toho deterministicky vygeneruje strukturovaný diagnostický incident (obsahující stack trace, diff a stav kontextu) a eskaluje jej vývojáři k lidskému zásahu.

3 Praktická část

3.1 Cíl a rozsah systému DarkFactory

Praktickou částí práce je systém **DarkFactory** — agentní harness instalovatelný jako aplikace pro platformu GitHub (GitHub App). Cílem systému je v maximální možné míře automatizovat rutinní fáze softwarového vývoje: od příjmu a sémantické analýzy požadavku v GitHub Issues, přes technické plánování a generování zdrojového kódu, až po spuštění validačních testů a vystavení pull requestu.

Ačkoliv systém usiluje o co nejvyšší míru automatizace, v reálném inženýrském provozu jej nelze označit za plně autonomní. Jazykové modely jsou stochastické systémy, které nemohou nést konečnou architektonickou odpovědnost ani garantovat stoprocentní bezchybnost výstupu. DarkFactory proto funguje v kooperativním režimu s člověkem (*Human-in-the-loop*), kde je volnost agenta striktně ohraničena bezpečnostními mantinely řídicího harnessu a klíčové přechody podléhají explicitnímu schválení člověkem. Zdrojový kód systému je veřejně dostupný (11).

Systém má převzít kognitivně rutinní a mechanické kroky vývojového procesu: strukturovanou interpretaci zadání, návrh technické dekompozice, implementaci v kódu a opravu regresí detekovaných testy. Nemá nahradit lidské rozhodování o tom, co a proč se má stavět; tato role zůstává člověku a harness je záměrně koncipován tak, aby si lidskou autorizaci vyžádal dříve, než provede zásadní či nevratné změny v repozitáři.

3.2 Architektura a životní cyklus požadavku

Systém DarkFactory je navržen jako modulární stavebnice složená z řídicích orchestračních modulů, sdílených šablon pracovních postupů pro kontinuální integraci a hermetického^{*} běhového prostředí. Architektura striktně odděluje deklarativní konfiguraci konkrétního repozitáře od samotné logiky orchestrace.

Životní cyklus požadavku formalizuje přechody mezi stavy v orientovaném acyklickém grafu (DAG) od zadání úkolu v GitHub Issues přes plánování, implementaci, automatické testování až po vystavení pull requestu a sloučení do hlavní větve.

Aby systém věděl, které validační a orchestrační úlohy má v repozitáři spustit, provádí automatickou detekci technologie na základě přítomnosti konfiguračních souborů balíčků:

^{*}Pojem **hermetické prostředí** (angl. *hermetic environment*) označuje v softwarovém inženýrství takové výpočetní a běhové prostředí, které je zcela izolované od nekontrolovaných stavů hostitelského operačního systému a okolní sítě. Veškeré nástroje, knihovny a systémové závislosti jsou v něm explicitně deklarovány a uzamčeny na konkrétních verzích, což zaručuje absolutní determinismus a reprodukovatelnost: proces spuštěný v hermetickém kontejneru skončí vždy identickým výsledkem bez ohledu na to, kde a kdy byl vyvolán.

Soubor	Prostředí	Doména
<code>pyproject.toml</code>	Python	kód
<code>Cargo.toml</code>	Rust	kód
<code>package.json</code>	Node	kód
<code>go.mod</code>	Go	kód
<code>typst.toml</code>	Typst	text
<code>.latexmkrc</code>	LaTeX	text
<code>lakefile.toml</code>	Lean	matematika

Tabulka 1: Rozpoznávání prostředí podle souboru popisujícího balíček.

Rozlišení dvou úrovní se ukázalo jako nutné v průběhu vývoje: *prostředí* určuje, jaké konkrétní binární nástroje jsou vyžadovány (např. Python vs. Rust), zatímco *doména* specifikuje způsob řízení výstupu (např. kompilace a testy v doméně kódu oproti sazbě a generování PDF v doméně textu). Toto rozlišení umožňuje obsloužit repozitář kombinující programový kód se sazební dokumentací. Konfigurační manifest navíc umožňuje automaticky detekované parametry deklarativně přepsat či doplnit (např. o specifické cesty k písmům).

3.3 Orchestrační jádro a validační infrastruktura

Orchestrační jádro systému řeší čtyři klíčové provozní výzvy autonomního běhu:

- **Orchestrace napříč poskytovateli a rotace modelů:** Systém neváže vývoj na jediné komerční rozhraní. Úkoly jsou popsány nezávisle na formátu konkrétního poskytovatele. Pokud primární model vyčerpá kvótu nebo je nedostupný, harness automaticky předá stav navazujícímu modelu v rotačním žebříčku, což eliminuje prostoje pipeline způsobené limity API.
- **Deterministické ověřování změn:** Každá změna podléhá požadovaným kontrolám v CI pipeline. Úlohy jsou navrženy tak, aby vždy deterministicky skončily explicitním výstupem a zamezily permanentnímu zablokování větve v důsledku přeskočených kontrol.
- **Projektová automatizace a správa nastavení jako kód:** Veškeré nastavení repozitáře, pravidla ochrany větví i projektové nástěnky (GitHub Projects) jsou spravovány deklarativně, což zaručuje plnou auditovatelnost a reprodukovatelnost konfigurace.
- **Automaticky odvozovaná živá dokumentace (*Living Documentation*):** V moderním softwarovém vývoji představuje manuální údržba dokumentace trvalý zdroj desynchronizace. DarkFactory proto prosazuje odvozování veškeré dokumentace a specifikací přímo ze zdrojového kódu a strukturovaných inline komentářů s automatickou validací v CI.

3.4 Systém revizních značek pro lidský dohled nad akademickým textem

Při rozšiřování systému DarkFactory na tvorbu a revizi odborných textů (doména textu) vyvstala potřeba formalizovat spolupráci člověka a agenta přímo v sazebním formátu Typst. Výsledkem je protokol vizuálních revizních značek (**Review Markers**) a textových revizních funkcí, který barevně a sémanticky rozlišuje stav zpracování jednotlivých pasáží: V toku textu se uplatňují tyto zvýrazňovací a srovnávací funkce:

- **Neověřený koncept** (`#draft[...]` / `#unconfirmed[...]`): Označuje neověřený text konceptu čekající na autorské posouzení a revizi.
- **Nově přidaný text** (`#added[...]`): Označuje nově přidaný text vygenerovaný autonomním agentem na základě požadavku či doporučení.
- **Potvrzený text** (`#confirmed[...]`): Označuje text potvrzený uživatelem, který dosud neprošel finální integrací.
- **Navrženo k odstranění** (`#removed[...]`): Označuje text navržený k odstranění z rukopisu.
- **Srovnávací diff** (`#diff(old, new)`): Zobrazuje původní text přeškrtnutý v červené barvě následovaný novým textem v zelené barvě.
- **Čistý neoznačený text**: Představuje finální, autorsky schválený a přijatý text v hlase autora.

Tento protokol umožňuje autonomnímu agentovi navrhopvat změny s transparentním vyznačením míry jistoty a člověku poskytuje okamžitou vizuální kontrolu nad tím, které části rukopisu již prošly lidskou redakcí a které ještě čekají na posouzení.

4 Výsledky a diskuse

4.1 Sjednocení pracovních postupů a přenositelnost napříč doménami

Základním architektonickým principem navrženého harnessu je centralizované sdílení pracovních postupů namísto jejich ad-hoc kopírování do jednotlivých projektů. Klientské repozitáře neudržují vlastní izolované sady validačních skriptů, linterů ani integračních definic; namísto toho delegují exekuci na centrální znovupoužitelné šablony a veškerou projektovou specifičnost deklarují v jediném manifestu.

Skutečný inženýrský přínos sjednocení spočívá v kvalitativních a architektonických vlastnostech:

- **Údržbová složitost $O(1)$:** Před sjednocením vyžadovala jakákoli oprava či bezpečnostní aktualizace v CI procesu samostatnou manuální úpravu a pull request v každém projektu zvlášť ($O(N)$). V centralizovaném modelu je úprava provedena pouze jednou v řídicím harnessu a spotřebitelské projekty ji přebírají posunem připnuté verze v manifestu.
- **Statická kontrola a testovatelnost:** Centralizované jádro harnessu podléhá striktní typové kontrole a je pokryto sadou jednotkových testů.
- **Omezení jediného bodu selhání (SPOF):** Riziko nechtěných či nekompatibilních změn je deterministicky eliminováno verzováním: projekty odkazují na neměnný kryptografický SHA hash commitu či sémantický tag.
- **Konzistentní agentní prostředí:** Zajištění, že životní cyklus požadavku, správa kontextu, rotace poskytovatelů i vyhodnocování validačních bran probíhají ve všech projektech exaktně stejným způsobem.

Klíčovou vlastností modulární architektury je striktní oddělení domény od prostředí. Zatímco prostředí specifikuje konkrétní sadu exekučních binárek (např. Python, Rust, Typst), doména vymezuje způsob řízení a verifikace výstupu:

- **Doména programového kódu:** Výstup podléhá kompilaci, statické analýze a exekuci jednotkových testů s měřením pokrytí.
- **Doména textu a dokumentace:** Výstup podléhá sazební kompilaci, kontrole terminologie a generování statických auditovatelných artefaktů (např. PDF dokumentace).

Díky deklarativnímu konfiguračnímu modelu dokáže řídicí harness obsluhovat obě domény identickým orchestračním automatem, aniž by bylo nutné duplikovat integrační infrastrukturu či větvit agentní smyčky.

4.2 Provozní metriky a spolehlivost agentních běhů

4.3 Poznatky a systémová úskalí z provozu

Praktické zkušenosti s během autonomních agentů v CI odhalily, že nejkritičtější úskalí neleží v neschopnosti modelů generovat syntakticky správný kód, nýbrž v distribuovaném řízení a systémové orchestraci:

- **Řízení souběžnosti a zámky:** Koordinace volajících a volaných úloh vyžaduje striktní hierarchizaci a zámky na úrovni větví, aby nedocházelo k uváznutí workflow nebo kolizím při souběžných integračních bězích.
- **Transparentnost hlášení selhání:** Systém nesmí tlumit výjimky a maskovat chyby; každé selhání API nebo nástroje musí být transparentně zaznamenáno v kontextu a eskalováno. Tiché maskování chyb vytváří iluzi stability a vede k rozsáhlým halucinacím modelu.
- **Dynamická detekce prostředí:** Systém nesmí spoléhat na implicitní předpoklady o technologiích v repozitáři, ale odvozovat exekuční plán z deklarativních manifestů.

4.4 Diskuse: Porovnání architektur

Zásadní otázkou je, jak si navržená architektura řídicího harnessu stojí ve srovnání se současnými agentními vývojovými platformami (např. SWE-agent (8), Devin či GitHub Copilot Workspace):

- **Deterministický DAG vs. nekonečná agentní smyčka:** Systémy jako SWE-agent spouštějí jediný monolitický model v interaktivní terminálové smyčce, kde model sám rozhoduje o ukončení práce. Pokud model uvízne v bludném kruhu nebo halucinuje, snadno vyčerpá celý rozpočet tokenů. Navržený harness naproti tomu uzavírá model do deterministického orientovaného acyklického grafu (DAG): každý krok (plánování, implementace, verifikace) má striktně alokovaný rozpočet tahů a samostatnou kontextovou izolaci.
- **Granularita lidského dohledu:** Většina komerčních nástrojů staví na paradigmatu „jedno zadání → finální pull request“. Pokud agent na začátku špatně pochopí záměr, vygeneruje stovky řádků nepoužitelného kódu, jehož následná revize vývojáře vyčerpává (*review fatigue*). Harness toto riziko eliminuje vícefázovým schvalováním (Human Gate): člověk autorizuje záměr a technický plán dříve, než je spuštěn samotný kódovací agent.
- **Bezpečnost a hermetičnost:** Běh autonomního agenta přímo na nechráněném vývojovém stroji přináší riziko poškození konfigurace nebo úniku tajemství. Navržený harness prosazuje exekuci v efemérních kontejnerech CI platformy s oddělenými oprávněními tokenů a auditovatelným protokolem každého provedeného příkazu.

4.5 Systémová a metodická omezení

Dosažená zjištění je třeba interpretovat s ohledem na tři fundamentální omezení:

1. **Hranice deterministické verifikovatelnosti:** Systém vyžaduje, aby bylo správnost navržené změny možné objektivně ověřit automatizovanými testy, lintery či kompilátorem. U úloh subjektivní či kreativní povahy — jako je ergonomie uživatelského rozhraní, ladění vizuálních stylů nebo stylistická formulace odborného textu — zůstává autonomní přínos omezen na vygenerování prvotního návrhu, jehož validaci musí provést člověk.
2. **Propustnost a dostupnost inferenčních API:** Ačkoli rotační žebříček poskytovatelů minimalizuje dopad výpadku jedné služby, celková průchodnost pipeline je limitována globálními kvótami a latencí cloudových API. Při souběžném zpracování většího množství požadavků může dojít k vyčerpání všech dostupných účtů, což pipeline dočasně pozastaví.
3. **Absence vícevláknového řešení konfliktů:** Architektura se soustředí na deterministické řešení izolovaných požadavků. Řízení komplexních merge konfliktů a vzájemných interferencí při souběžné práci velkého množství lidských vývojářů a autonomních agentů nad stejnými větvemi zůstává otevřenou výzvou pro navazující výzkum.

5 Závěr

Seznam příloh

A	Obsah přiloženého média	24
B	Schéma konfiguračního manifestu darkfactory.json	25
C	Sdílené workflow pro GitHub Actions	26
D	Systémové prompty plánovacího a kódovacího agenta	27
E	Protokol revizních značek v sazebním systému Typst	28
E.1	Vizuální reprezentace jednotlivých prvků v sazbě	28

A Obsah přiloženého média

Odevzdaný archiv obsahuje kompletní zdrojové soubory práce, sazební šablonu, řídicí skripty a zdrojový kód autonomního systému DarkFactory.

```
mono-OdbornaPrace/
├─ prace/                                # Rukopis a sazba odborné práce
│   └─ kapitoly/                         # Texty jednotlivých kapitol (01-uvod
až 06-prilohy)
│   └─ lib/odborna-prace.typ            # Sazební šablona, recenzní značky a
formátování GJKT
│   └─ img/                             # Vektorová procesní schémata (SVG) a
grafické podklady
│   └─ fonts/                           # Metricky shodná patková písma
(Caladea)
│   └─ bib/references.bib               # Bibliografická databáze citovaných
zdrojů
│   └─ scripts/                         # Pomocné a náhledové skripty
│   └─ main.typ                         # Hlavní řídicí dokument sazby
│   └─ metadata.typ                     # Údaje o autorovi, vedoucím a
anotace
│   └─ Makefile                         # Příkazy pro automatizovanou
kompilaci a kontrolu
├─ darkfactory/                         # Zdrojový kód systému DarkFactory
(submodul)
│   └─ .github/workflows/               # Sdílené šablony kontinuální
integrace
│   └─ src/                             # Výkonné orchestrační jádro systému
├─ out/prace.pdf                        # Výsledný vysázený tiskový dokument
v PDF
└─ README.md                           # Dokumentace a návod na reprodukci
prostředí
```

Výpis 1: Stromová adresářová struktura odevzdaného elektronického archivu a doprovodných repozitářů.

B Schéma konfiguračního manifestu darkfactory.json

C Sdílené workflow pro GitHub Actions

D Systémové prompty plánovacího a kódovacího agenta

E Protokol revizních značek v sazebním systému Typst

Tato příloha uvádí referenční definici a použití vizuálních revizních značek pro řízení a dohled nad generovaným textem v sazebním formátu Typst.

```
#import "lib/odborna-prace.typ": note, issue, alert, critique, added,
draft, confirmed, diff

// --- 1. Panely na okraji textu (Callouty) ---
#note[Doplňte porovnání rychlosti kompilace mezi verzemi 0.1 a 0.2.]
#issue[Chybná signatura funkce: chybí povinný parametr timeout.]
#alert[Sekce postrádá shrnutí naměřených výsledků před diskusí.]
#critique[Metodologická absence baseline prokazující přínos nového
modulu.]
#blue-note[Metodické vymezení: rozsah práce a oddělení agentního
inženýrství od strojového učení.]

// --- 2. Textové revizní funkce (zvýraznění v toku textu) ---
#draft[Tento odstavec tvoří neověřený koncept čekající na schválení.]
#added[Nově vygenerovaná sekce automaticky začleněná agentem.]
#confirmed[Uživatelem zkontrolovaný text, který ještě nebyl finalizován.]
#removed[Zastaralý text navržený k odstranění.]
#diff[Původní chybné znění textu.][Nové opravené znění textu po revizi.]
```

Výpis 2: Ukázka zápisu a použití revizních značek a textových funkcí v jazyce Typst.

E.1 Vizuální reprezentace jednotlivých prvků v sazbě

Pro přehlednost jsou níže uvedeny reálné ukázky jednotlivých revizních panelů a textových funkcí v jejich finální vizuální podobě:

Ukázky textových zvýrazňovacích a srovnávacích funkcí v toku odstavce:

- **Neověřený koncept** (`#draft` / `#unconfirmed`): Tento text představuje koncept čekající na posouzení autorem.
- **Nově přidáný text** (`#added`): Tato pasáž byla nově vygenerována autonomním agentem na základě požadavku.
- **Potvrzený text** (`#confirmed`): Text byl předběžně odsouhlasen uživatelem, čeká na finální začištění.
- **Navrženo k odstranění** (`#removed`):
- **Srovnávací diff** (`#diff`): Nové přesné, fakticky a formálně ověřené znění pasáže.
- **Čistý neoznačený text**: Představuje finální, autorsky schválený text v hlase autora bez jakéhokoliv podbarvení.

Každá značka plní přesně vymezenou komunikační roli v procesu lidského schvalování: zatímco finální text zůstává zcela bez zvýraznění, veškeré neověřené pasáže konceptu jsou

zřetelně žluté (`#draft` / `#unconfirmed`), nově přidané části zelené (`#added`), potvrzené části modré (`#confirmed`) a opravy zviditelněné přes srovnávací diff (`#diff`). Náměty, chyby, strukturální vady i břitká kritika jsou navíc striktně separovány do barevných postranních panelů na okraji textu.

Seznam zdrojů

1. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. IS-BN 978-1-4842-0076-6.
2. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
3. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
4. XIAO, Guangxuan, TIAN, Yuandong, CHEN, Beidi, HAN, Song a LEWIS, Mike. Efficient Streaming Language Models with Attention Sinks. *arXiv preprint arXiv:2309.17453*. Online. 2023. Available from: <https://arxiv.org/abs/2309.17453>
5. ZHANG, Zhenyu, SHENG, Ying, ZHOU, Tianyi, CHEN, Tianlong, ZHENG, Lianmin, CAI, Ruisi, SONG, Zhao, TIAN, Yuandong, RÉ, Christopher, BARRETT, Clark, WANG, Zhangyang a CHEN, Beidi. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 34661–34730. Available from: <https://arxiv.org/abs/2306.14048>
6. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
7. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
8. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
9. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
10. Meta Harness: End to end optimization of Agent Harnesses. *arXiv preprint arXiv:2603.28052*. Online. 2026. Available from: <https://arxiv.org/abs/2603.28052>
11. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
12. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>

13. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
14. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>